CS & IT ENGINEERING



Algorithms

Analysis of Algorithms



Lecture No.- 02

By- Aditya sir

Recap of Previous Lecture



Topic — Intro about Course

Topic — Intro to Algo

→ why
what
☆ How of analysis

Topics to be Covered



- Topic
- Topic
- Topic

How to analyse

Apriori Aposteriori

Adv, disadv



About Aditya Jain sir

HW

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

TTTe



Topic : Analysis of Algorithms

How to Analyses?

Methodology of Analysis

- Aposteriori
  After implementation
- Apriori
  Before implementation



Topic : Analysis of Algorithms

2. Apriori Analysis

Carried out before the actual implementation of Algorithm.



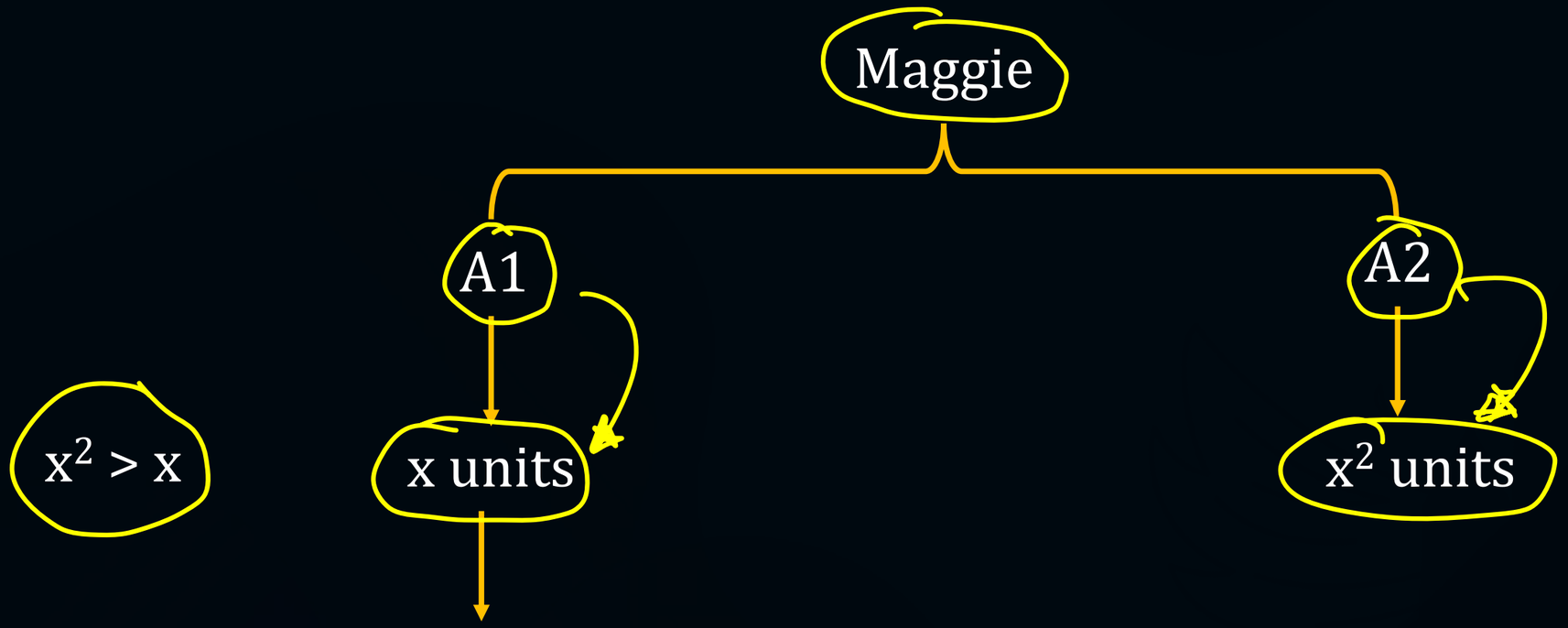
Topic : Analysis of Algorithms

Advantages	Disadvantages
1. Platform independent	1. It gives approximate value
2. Can be carried out without actual implementation	(Time complexity)
3. Helps us to compare the relative performance of 2 or more algorithms.	(Space complexity)



Topic : Analysis of Algorithms

Eg.1.



Comparison can be made
With approximate values



Topic : Analysis of Algorithms

- Steps to carry out Apriori Analysis:
  - Pseudo code of the algorithm is required
  - Need a matrix to compare tow or more algorithm running time.
  - Need some notation/ symbol to represent the running time.

Asymptotic



Topic : Analysis of Algorithms

- Apriori Analysis can be carried out in 2 ways.

1. Step- count method → exact [illegible] operations (not much used)
2. Order of magnitude method → give approximate no. of operations.

Assumption:- Every fundamental operation takes..

1 unit of time.

$p + q$

$p = (qr + \gamma)$ — 2 unit

$i : 1 \longrightarrow 1$

n update $\longrightarrow n$

$(i \le n)$

condition $\longrightarrow (n+1)$ units

```
for i=1; i<=n; i++
{ a = b + c }   2
}
```

$1 + n + (n+1) + 2n$

$= (4n + 2)$ units

$n = 3$

$i = 1$

$i = 2$

$i = 3$

$i = 4$

$i \le 3$

$i \le 3$

$i \le 3$

$i \le 3$ Stop



Topic : Analysis of Algorithms

1. Step-count method:-

```
Algo. AJSir(n) {
    1. p = q + r          → 2units
    2. x = y * z          → 2units
    3. for (i = 1, i <= n ; i++) {      → (4n + 2) units
            a = b + c
       }
    4. for (j = 1; j <= n, j++) {
            for (k = 1;  k  < = n; k ++) {
                x =  y + z
            }
       }
}
```

→ $(4n^2 + 4n + 2)$ units

optional HW



Topic : Analysis of Algorithms

Eg.2. for (i =1; i <=n; i++)→ (2n+2)

{

a = b +c → (2*n)

}

1. initialization i =1
2. Condition i <=n
3. updation i = i +1

eg →

Total operation = (2n + 2) + 2*n

= 4n + 2

for (i =1; i <=4; i++)

i = 1 → 1	1 → 2
i <=4 → Four	2 → 2
i++→ 2	3 → 2
	4 → 2
	5× → 1



Topic : Analysis of Algorithms

For (i=1, i<=n, i++)

1 → initialization

(n +1) → comparison

n → updation

1 + (n +1) +n = (2n+2)



Topic : Analysis of Algorithms

Bonus

runs n time → `for (j =1; j<=n; j++)` → (2n+2)

```
{
```

runs n time → `for (k =1; k<=n; k++)` → (2n+2)

```
    {
        x = y + z → 2
    }
}
```

n^2 times

Total operation

$(2n + 2) + n*(2n+2) + n^2*2$ — $2 \times n^2$

$\rightarrow 2n^2 + 2n + 2n + 2 + 2n^2$

$\rightarrow (4n^2 + 4n + 2)$



Topic : Analysis of Algorithms

```
Algo AJSir(n)
{
        .....
        .....
        .....
}
```

Total no. of operation
$= 2 + 2 + (4n+2) + (4n^2 + 4n + 2)$
$= 4n^2 + 8n + 8$ units of times

Step-Count Approach.

not useful

$4n^2 + 8n + 8$

Startup

yr1 → 2026 → 5 emp

yr2 → 2027 → 20 emp

→ 2030 → 100 emp

→ 2040 → 10000 emp

→ 2050 → 500000 emp → interview

SL + dominating

☆☆



Topic : Analysis of Algorithms

2) Approximate - Order of Magnitude ★

(independent of n) → we just need to consider the order of the number of fundamental operation in the fundamental statement.

→ 1
Constant

Linear → n

Quadratic → n^2

$4n^2 + 2 \Longrightarrow n^2$

Topic : Analysis of Algorithms

Order of Magnitude approach.

1. ~~Step count method:-~~

```
Algo. AJSir(n) {
    1.  p = q + r          → 1 or 2   → constant →1
    2.  x = y * z          → 1 or 2   → constant →1
    3.  for (i = 1, i <= n ; i++) {   → n units
            a = b + c
        }
    4.  for (j = 1; j <= n, j++) {
            for (k = 1;  k  <= n; k ++) {
                x =  y + z
            }
        }
}
```

$4n + 2 \longrightarrow n$

$\rightarrow n^2$ units

$4n^2 + 4n + 2$



Topic : Analysis of Algorithms

Total units as per order of Magnitude approach

$\Rightarrow \quad 1 + n + n^2$

$\Rightarrow \quad (n^2 + n + 1)$

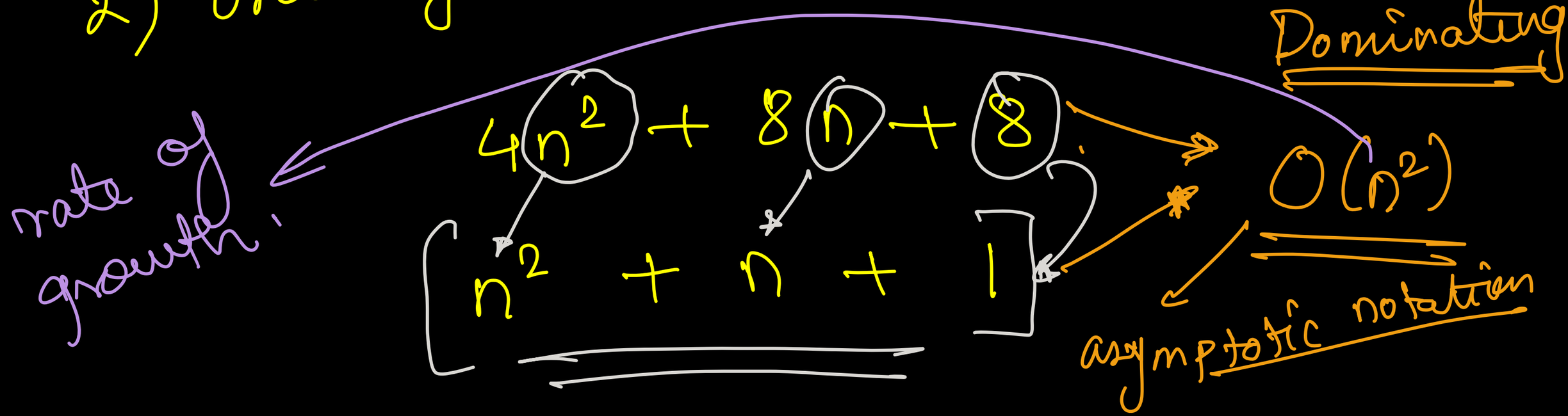
for same algorithm the number of units by step- count method $\rightarrow (4n^2 + 8n + 8)$

$\Rightarrow$ notation = $O(n^2)$

Same Algo Algo A.T.C)

1) Step-Count mtd ⟶ $(4n^2 + 8n + 8)$ units

2) Order of Magnitude ⟶ $n^2 + n + 1$





Topic : Analysis of Algorithms

Note.

Aim of apriori analysis is to get/represent the running time of on algo. As a mathematical function of input size 'n'.

E.g.3.

$T(n) = 2n^2 + 2$

$T(n) = 4n^2 + 8n + 8$

$T(n) = 5n^2 + 2$

Order of magnitude refers to the rate of growth of time w.r.t. 'n'

$O(n^2)$



Topic : Analysis of Algorithms

Rate of growth of time as a mathematical function

Logarithmic

⇓

Form: Log(n)

E.g. $\log_5(n)$, $\log_2(n)$

log(log(n)

log n

Polynomial

n^n

⇓

Constant ↑ n^0, n^1, $n^{0.5} \Rightarrow \sqrt{n}$, n^2 → quadratic ← Sq.root

n^3

↓

cubic

linear

N* log(n)

Exponential

Exponential

⇓

x^2

n^n

E.g.

3^n, 2^n, 7^n...

n^n, n!

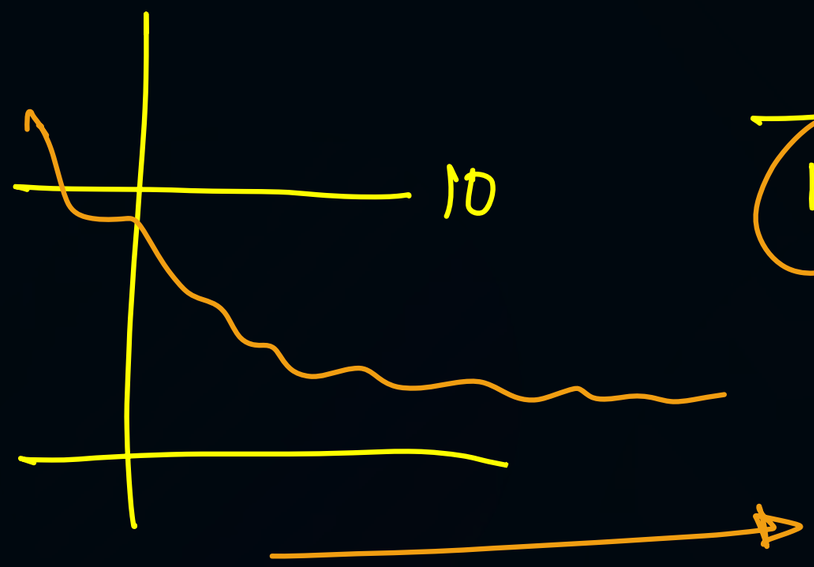
3^n 2^n 7^n, n^n

Topic : Analysis of Algorithms

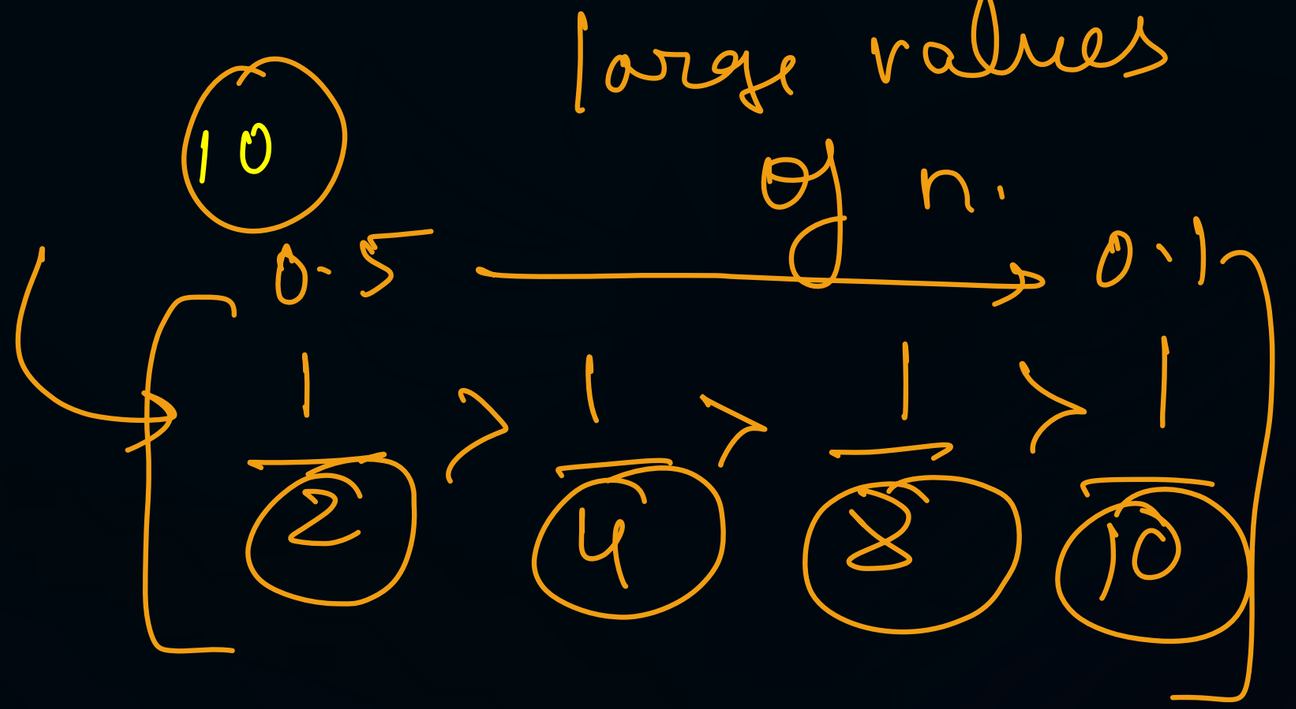
Note. Rate of growth comparison [in general]

Usually [Decreasing function < Constant < logarithm < Polynomial < Exponential]

E.g.4. $\left[\left(\frac{1}{n}\right) < 10 < \log_2\sqrt{n} < n < n^2 \ldots 2^n < 3^n .. n^n\right]$









Topic : Analysis of Algorithms

How to compare the rate of growth of 2 function?

Larger input size (n)

(Linear) (n)	(expo) 2^n	(quadratic) n^2
1	2	1
2	4	4
3	8	9
4	16	16
5	32	25
6	64	36
7	128	49

($2^n > n^2$ for larger values of n)

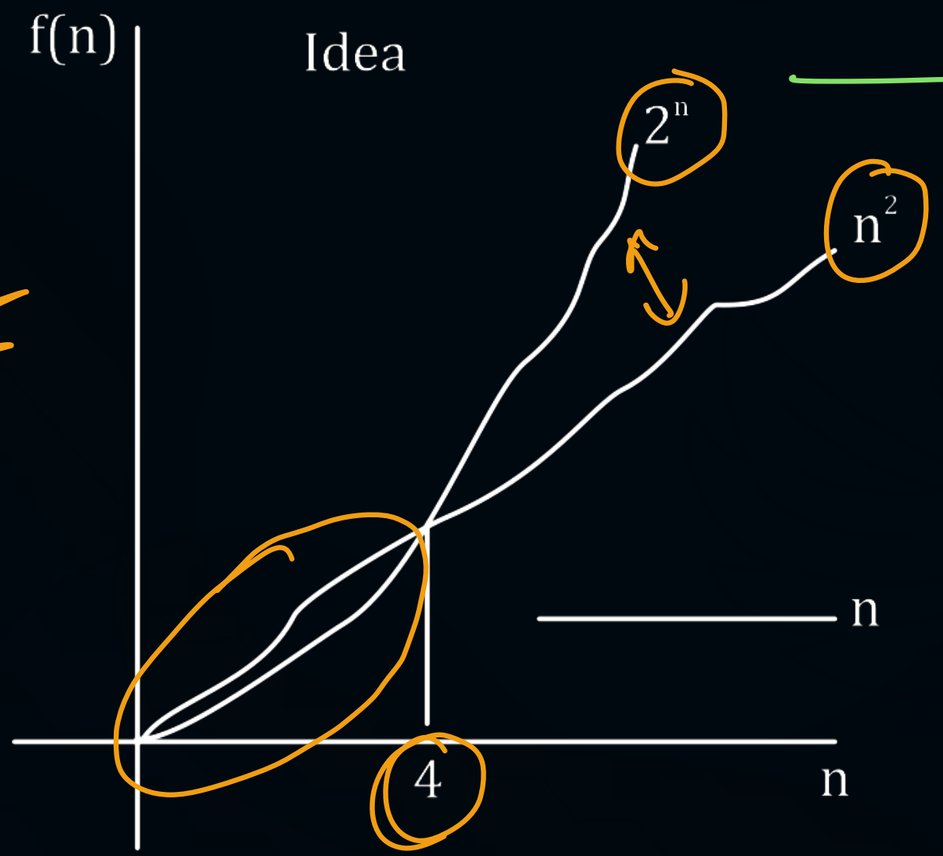


Topic : Analysis of Algorithms

(Poly vs Expo)

Imp :- Compare for largeer values of n.

Poly < Expo





Topic : Analysis of Algorithms

Algo→ n^2 → Lower rate of growth → takes less time → better

Algo→ 2^n → High rate of growth → takes more time → not good.

Note:-

[Algorithm that takes polynomial unit of time all are more efficient that those with exponential time.]



Topic : Analysis of Algorithms

- Apriori Analysis

$(n^2 + n+1)$ → order of magnitude

1. Time complexity/ Running time as a function of 'n' → (input size)
2. To understand the change in nature or running time for a given algo and a fixed input size 'n' but for different input classes. → (test cases)

Best case (Bc)

Average case (Ac)

Worst case (Wc)

O(n)

Algo i/p: n=5

n=100000

n=5

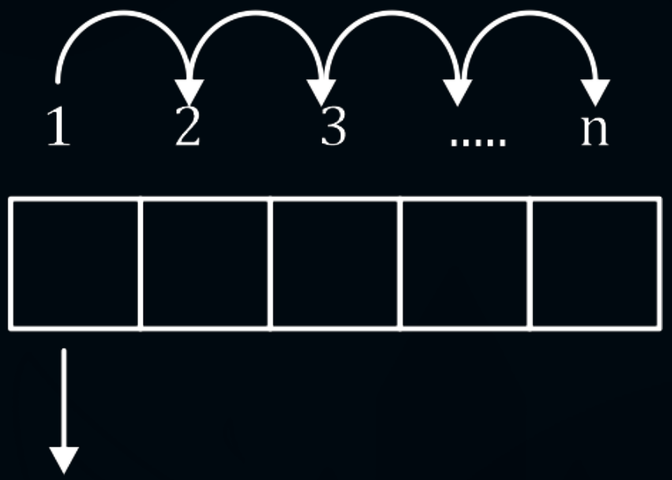
n=5



Topic : Analysis of Algorithms

Example to understand point (2)

```
Algorithm linear search (A, n key)
{
        for (i = 1, i <=n, i++)
        {
                if (A[i] == key)
                        return I
        }
}       Print ("element not fund")
```



n = 5

A[5] , key



Topic : Analysis of Algorithms

Eg.5. n = 5 (fixed)

Case1:- input case-1



Loop runs how many times?

⇒ 1 time → Constant

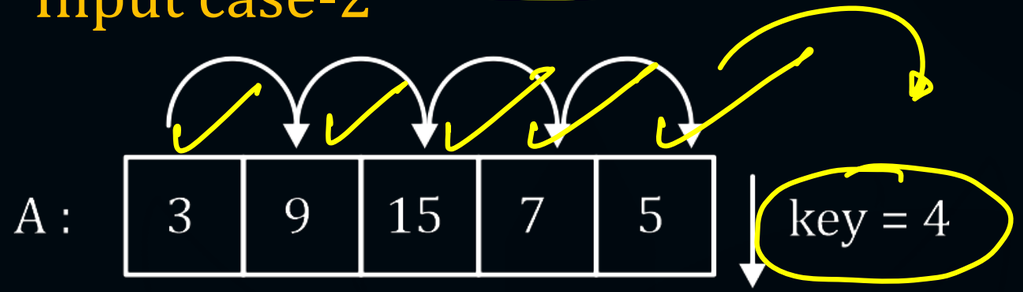
Best case TC ⇒ TC ⇒ O(1)

independent of 'n'

Case2:-

Key is not present or is present at last position

Input case-2



→ Loop runs how many times?

⇒ 5 time →(n times)

Linear Rate of growth

Worst case TC of ⇒ TC= O(n) linear search



Topic : Analysis of Algorithms

V.V.V.Imp

- Different cases for time complexity for a fixed input size.

1. Best case → The input for which the algo. Runs min no. of times (does min work.)

   The time complexity for such input ⇒ Best case TC

2. Worst case→ The input which of the algorithm run max no. of times

   The time complexity for such input ⇒ worst case TC

3. Average case→ deals with probability.

Ex. Linear search

BC TC → O(1) | WC TC → O(n) | AC TC → O(n) → Bonus.



Topic : Analysis of Algorithms

For any given algo, in general:

Always: $B(n) \leq A(n) \leq W(n)$

Best case TC → B(n)
Worst case TC → W(n)
Avg. case TC → A(n)

Examples:

HW

1. $B(n) < [A(n) = W(n)] \Rightarrow$ Linear search, Binary search
2. $[B(n) = A(n)] < W(n) \Rightarrow$ Quick sort
3. $[B(n) = A(n) = W(n)] \Rightarrow$ Merge sort, Heap sort, Selection sort.



Summary

1) Apriori Analysis
   - Step-count mtd
   - Order of Magnitude



2) Types of Cases
   1) Best Case
   2) Worst Case



THANK - YOU